

# UNIT-III

Exploring Java Language, Collections Overview, Collections Interfaces, Collections Classes, Iterators, Random Access Interface, Maps, Comparators, Arrays, Legacy classes and interfaces, Sting Tokenizer, BitSet, Date, Calendar, Timer.

## Collections in Java

The **Collection in Java** is a framework that provides an architecture to store and manipulate the group of objects.

Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.

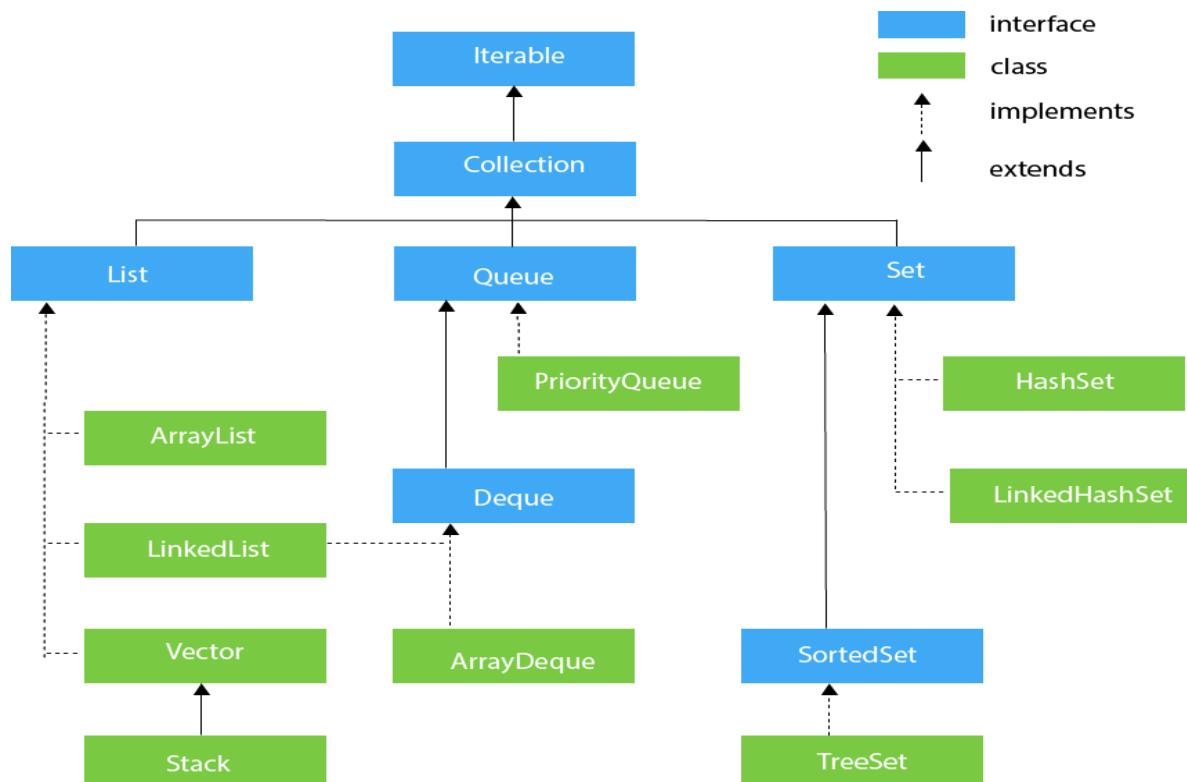
Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).

### *What is a framework in Java*

- It provides readymade architecture.
- It represents a set of classes and interfaces.
- It is optional.

## Hierarchy of Collection Framework

Let us see the hierarchy of Collection framework. The **java.util** package contains all the classes and interfaces for the Collection framework.



# Methods of Collection interface

There are many methods declared in the Collection interface. They are as follows:

| No. | Method                                                | Description                                                                                     |
|-----|-------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 1   | public boolean add(E e)                               | It is used to insert an element in this collection.                                             |
| 2   | public boolean addAll(Collection<? extends E> c)      | It is used to insert the specified collection elements in the invoking collection.              |
| 3   | public boolean remove(Object element)                 | It is used to delete an element from the collection.                                            |
| 4   | public boolean removeAll(Collection<?> c)             | It is used to delete all the elements of the specified collection from the invoking collection. |
| 5   | default boolean removeIf(Predicate<? super E> filter) | It is used to delete all the elements of the collection that satisfy the specified predicate.   |
| 6   | public boolean retainAll(Collection<?> c)             | It is used to delete all the elements of invoking collection except the specified collection.   |
| 7   | public int size()                                     | It returns the total number of elements in the collection.                                      |
| 8   | public void clear()                                   | It removes the total number of elements from the collection.                                    |
| 9   | public boolean contains(Object element)               | It is used to search an element.                                                                |

|    |                                                |                                                                                                                 |
|----|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| 10 | public boolean<br>containsAll(Collection<?> c) | It is used to search the specified collection in the collection.                                                |
| 11 | public Iterator iterator()                     | It returns an iterator.                                                                                         |
| 12 | public Object[] toArray()                      | It converts collection into array.                                                                              |
| 13 | public <T> T[] toArray(T[] a)                  | It converts collection into array. Here, the runtime type of the returned array is that of the specified array. |
| 14 | public boolean isEmpty()                       | It checks if collection is empty.                                                                               |
| 15 | default StreamparallelStream()                 | It returns a possibly parallel Stream with the collection as its source.                                        |
| 16 | default Stream stream()                        | It returns a sequential Stream with the collection as its source.                                               |
| 17 | default Spliteratorspliterator()               | It generates a Spliterator over the specified elements in the collection.                                       |
| 18 | public boolean equals(Object element)          | It matches two collections.                                                                                     |
| 19 | public int hashCode()                          | It returns the hash code number of the collection.                                                              |

## Iterator interface

Iterator interface provides the facility of iterating the elements in a forward direction only.

### *Methods of Iterator interface*

There are only three methods in the Iterator interface. They are:

| No. | Method | Description |
|-----|--------|-------------|
|-----|--------|-------------|

|   |                             |                                                                                  |
|---|-----------------------------|----------------------------------------------------------------------------------|
| 1 | public boolean<br>hasNext() | It returns true if the iterator has more elements<br>otherwise it returns false. |
| 2 | public Object next()        | It returns the element and moves the cursor pointer to<br>the next element.      |
| 3 | public void remove()        | It removes the last elements returned by the iterator. It<br>is less used.       |

## Iterable Interface

The Iterable interface is the root interface for all the collection classes. The Collection interface extends the Iterable interface and therefore all the subclasses of Collection interface also implement the Iterable interface.

## Collection Interface

The Collection interface is the interface which is implemented by all the classes in the collection framework. It declares the methods that every collection will have. In other words, we can say that the Collection interface builds the foundation on which the collection framework depends.

Some of the methods of Collection interface are Boolean add ( Object obj), Boolean addAll ( Collection c), void clear(), etc. which are implemented by all the subclasses of Collection interface.

## List Interface

List interface is the child interface of Collection interface. It inhibits a list type data structure in which we can store the ordered collection of objects. It can have duplicate values.

List interface is implemented by the classes ArrayList, LinkedList, Vector, and Stack.

To instantiate the List interface, we must use :

1. List <data-type> list1= **new** ArrayList();
2. List <data-type> list2 = **new** LinkedList();
3. List <data-type> list3 = **new** Vector();
4. List <data-type> list4 = **new** Stack();

## ArrayList

The ArrayList class implements the List interface. It uses a dynamic array to store the duplicate element of different data types. The ArrayList class maintains the insertion order and is non-synchronized. The elements stored in the ArrayList class can be randomly accessed. Consider the following example.

```

import java.util.*;
class TestJavaCollection1{
public static void main(String args[]){
ArrayList<String> list=new ArrayList<String>();//Creating arraylist
list.add("Ravi");//Adding object in arraylist
list.add("Vijay");
list.add("Ravi");
list.add("Ajay");
//Traversing list through Iterator
Iterator itr=list.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}

```

## LinkedList

LinkedList implements the Collection interface. It uses a doubly linked list internally to store the elements. It can store the duplicate elements. It maintains the insertion order and is not synchronized. In LinkedList, the manipulation is fast because no shifting is required.

Consider the following example.

```

import java.util.*;
public class TestJavaCollection2{
public static void main(String args[]){
LinkedList<String> al=new LinkedList<String>();
al.add("Ravi");
al.add("Vijay");
al.add("Ravi");
al.add("Ajay");
Iterator<String> itr=al.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}

```

## Vector

Vector uses a dynamic array to store the data elements. It is similar to ArrayList. However, It is synchronized and contains many methods that are not the part of Collection framework.

Consider the following example.

```

import java.util.*;
public class TestJavaCollection3{
public static void main(String args[]){
Vector<String> v=new Vector<String>();
v.add("Ayush");
v.add("Amit");
v.add("Ashish");
v.add("Garima");
Iterator<String> itr=v.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}

```

## Stack

The stack is the subclass of Vector. It implements the last-in-first-out data structure, i.e., Stack. The stack contains all of the methods of Vector class and also provides its methods like boolean push(), boolean peek(), boolean push(object o), which defines its properties.

Consider the following example.

```

import java.util.*;
public class TestJavaCollection4{
public static void main(String args[]){
Stack<String> stack = new Stack<String>();
stack.push("Ayush");
stack.push("Garvit");
stack.push("Amit");
stack.push("Ashish");
stack.push("Garima");
stack.pop();
Iterator<String> itr=stack.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
} }

```

## Queue Interface

Queue interface maintains the first-in-first-out order. It can be defined as an ordered list that is used to hold the elements which are about to be processed. There are various classes like PriorityQueue, Deque, and ArrayDeque which implements the Queue interface.

Queue interface can be instantiated as:

```
Queue<String> q1 = new PriorityQueue();  
Queue<String> q2 = new ArrayDeque();
```

## PriorityQueue

The PriorityQueue class implements the Queue interface. It holds the elements or objects which are to be processed by their priorities. PriorityQueue doesn't allow null values to be stored in the queue.

Consider the following example.

```
import java.util.*;  
public class TestJavaCollection5{  
    public static void main(String args[]){  
        PriorityQueue<String> queue=new PriorityQueue<String>();  
        queue.add("Amit Sharma");  
        queue.add("Vijay Raj");  
        queue.add("JaiShankar");  
        queue.add("Raj");  
        System.out.println("head:"+queue.element());  
        System.out.println("head:"+queue.peek());  
        System.out.println("iterating the queue elements:");  
        Iterator itr=queue.iterator();  
        while(itr.hasNext()){  
            System.out.println(itr.next());  
        }  
        queue.remove();  
        queue.poll();  
        System.out.println("after removing two elements:");  
        Iterator<String> itr2=queue.iterator();  
        while(itr2.hasNext()){  
            System.out.println(itr2.next());  
        }  
    }  
}
```

## Deque Interface

Deque interface extends the Queue interface. In Deque, we can remove and add the elements from both the side. Deque stands for a double-ended queue which enables us to perform the operations at both the ends.

Deque can be instantiated as:

```
Deque d = new ArrayDeque();
```

# ArrayDeque

ArrayDeque class implements the Deque interface. It facilitates us to use the Deque. Unlike queue, we can add or delete the elements from both the ends.

ArrayDeque is faster than ArrayList and Stack and has no capacity restrictions.

Consider the following example.

```
import java.util.*;
public class TestJavaCollection6{
public static void main(String[] args) {
    //Creating Deque and adding elements
    Deque<String> deque = new ArrayDeque<String>();
    deque.add("Gautam");
    deque.add("Karan");
    deque.add("Ajay");
    //Traversing elements
    for (String str : deque) {
        System.out.println(str);
    }
}
```

## Set Interface

Set Interface in Java is present in java.util package. It extends the Collection interface. It represents the unordered set of elements which doesn't allow us to store the duplicate items. We can store at most one null value in Set. Set is implemented by HashSet, LinkedHashSet, and TreeSet.

Set can be instantiated as:

1. Set<data-type> s1 = **new** HashSet<data-type>();
2. Set<data-type> s2 = **new** LinkedHashSet<data-type>();
3. Set<data-type> s3 = **new** TreeSet<data-type>();

## HashSet

HashSet class implements Set Interface. It represents the collection that uses a hash table for storage. Hashing is used to store the elements in the HashSet. It contains unique items.

Consider the following example.

```
import java.util.*;
```

```

public class TestJavaCollection7{
public static void main(String args[]){
//Creating HashSet and adding elements
HashSet<String> set=new HashSet<String>();
set.add("Ravi");
set.add("Vijay");
set.add("Ravi");
set.add("Ajay");
//Traversing elements
Iterator<String> itr=set.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}

```

## LinkedHashSet

LinkedHashSet class represents the LinkedList implementation of Set Interface. It extends the HashSet class and implements Set interface. Like HashSet, It also contains unique elements. It maintains the insertion order and permits null elements.

Consider the following example.

```

import java.util.*;
public class TestJavaCollection8{
public static void main(String args[]){
LinkedHashSet<String> set=new LinkedHashSet<String>();
set.add("Ravi");
set.add("Vijay");
set.add("Ravi");
set.add("Ajay");
Iterator<String> itr=set.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}

```

## SortedSet Interface

SortedSet is the alternate of Set interface that provides a total ordering on its elements. The elements of the SortedSet are arranged in the increasing (ascending) order. The SortedSet provides the additional methods that inhibit the natural ordering of the elements.

The SortedSet can be instantiated as:

```
SortedSet<data-type> set = new TreeSet();
```

## TreeSet

Java TreeSet class implements the Set interface that uses a tree for storage. Like HashSet, TreeSet also contains unique elements. However, the access and retrieval time of TreeSet is quite fast. The elements in TreeSet stored in ascending order.

Consider the following example:

```
import java.util.*;
public class TestJavaCollection9{
public static void main(String args[]){
    //Creating and adding elements
    TreeSet<String> set=new TreeSet<String>();
    set.add("Ravi");
    set.add("Vijay");
    set.add("Ravi");
    set.add("Ajay");
    //traversing elements
    Iterator<String> itr=set.iterator();
    while(itr.hasNext()){
        System.out.println(itr.next());
    }
}
```

## Java Collections class

Java collection class is used exclusively with static methods that operate on or return collections. It inherits Object class.

The important points about Java Collections class are:

- Java Collection class supports the **polymorphic algorithms** that operate on collections.
- Java Collection class throws a **NullPointerException** if the collections or class objects provided to them are null.

| SN | Modifier & Type    | Methods  | Descriptions                                                                  |
|----|--------------------|----------|-------------------------------------------------------------------------------|
| 1) | static <T> boolean | addAll() | It is used to adds all of the specified elements to the specified collection. |

|    |                                   |                       |                                                                                            |
|----|-----------------------------------|-----------------------|--------------------------------------------------------------------------------------------|
| 2) | static <T><br>Queue<T>            | asLifoQueue()         | It returns a view of a Deque as a Last-in-first-out (LIFO) Queue.                          |
| 3) | static <T> int                    | <u>binarySearch()</u> | It searches the list for the specified object and returns their position in a sorted list. |
| 4) | static <E><br>Collection<E>       | checkedCollection()   | It is used to returns a dynamically typesafe view of the specified collection.             |
| 5) | static <E> List<E>                | checkedList()         | It is used to returns a dynamically typesafe view of the specified list.                   |
| 6) | static <K,V><br>Map<K,V>          | checkedMap()          | It is used to returns a dynamically typesafe view of the specified map.                    |
| 7) | static <K,V><br>NavigableMap<K,V> | checkedNavigableMap() | It is used to returns a dynamically typesafe view of the specified navigable map.          |
| 8) | static <E><br>NavigableSet<E>     | checkedNavigableSet() | It is used to returns a dynamically typesafe view of the specified navigable set.          |

## Java Collections Example

```

import java.util.*;
public class CollectionsExample {
    public static void main(String a[]){
        List<String> list = new ArrayList<String>();
        list.add("C");
        list.add("Core Java");
        list.add("Advance Java");
        System.out.println("Initial collection value:"+list);
        Collections.addAll(list, "Servlet","JSP");
    }
}

```

```

        System.out.println("After adding elements collection value:"+list);
        String[] strArr = {"C#", ".Net"};
        Collections.addAll(list, strArr);
        System.out.println("After adding array collection value:"+list);
    }
}

```

## Iterators in Java

Iterators are used in Collection framework in Java to retrieve elements one by one. There are three iterators.

### 1. Enumeration :

It is a interface used to get elements of legacy collections(Vector, Hashtable). Enumeration is the first iterator present from JDK 1.0, rests are included in JDK 1.2 with more functionality. Enumerations are also used to specify the input streams to a *SequenceInputStream*. We can create Enumeration object by calling *elements()* method of vector class on any vector object

```
// Here "v" is an Vector class object. e is of
```

```
// type Enumeration interface and refers to "v"
```

```
Enumeration e = v.elements();
```

There are **two** methods in Enumeration interface namely :

```
// Tests if this enumeration contains more elements
```

```
public boolean hasMoreElements();
```

```
// Returns the next element of this enumeration
```

```
// It throws NoSuchElementException
```

```
// if no more element present
```

```
public Object nextElement();
```

```
// Java program to demonstrate Enumeration
```

```
import java.util.Enumeration;
```

```
import java.util.Vector;
```

```
public class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        // Create a vector and print its contents
```

```
        Vector v = new Vector();
```

```
        for (int i = 0; i < 10; i++)
```

```
            v.addElement(i);
```

```
        System.out.println(v);
```

```
        // At beginning e(cursor) will point to
```

```
        // index just before the first element in v
```

```
        Enumeration e = v.elements();
```

```
        // Checking the next element availability
```

```
        while (e.hasMoreElements())
```

```

        {
            // moving cursor to next element
            int i = (Integer)e.nextElement();

            System.out.print(i + " ");
        }
    }
}

```

Output:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
0 1 2 3 4 5 6 7 8 9
```

### Limitations of Enumeration :

- Enumeration is for **legacy** classes(Vector, Hashtable) only. Hence it is not a universal iterator.
- Remove operations can't be performed using Enumeration.
- Only forward direction iterating is possible.

## 2. Iterator:

It is a **universal** iterator as we can apply it to any Collection object. By using Iterator, we can perform both read and remove operations. It is improved version of Enumeration with additional functionality of remove-ability of a element.

Iterator must be used whenever we want to enumerate elements in all Collection framework implemented interfaces like Set, List, Queue, Deque and also in all implemented classes of Map interface. Iterator is the **only** cursor available for entire collection framework.

Iterator object can be created by calling *iterator()* method present in Collection interface.

```
// Here "c" is any Collection object. itr is of
```

```
// type Iterator interface and refers to "c"
```

```
Iterator itr = c.iterator();
```

Iterator interface defines **three** methods:

```
// Returns true if the iteration has more elements
```

```
public boolean hasNext();
```

```
// Returns the next element in the iteration
```

```
// It throws NoSuchElementException if no more
```

```
// element present
```

```
public Object next();
```

```
// Remove the next element in the iteration
```

```
// This method can be called only once per call
```

```
// to next()
```

```
public void remove();
```

*remove()* method can throw two exceptions

- *UnsupportedOperationException* : If the remove operation is not supported by this iterator
- *IllegalStateException* : If the next method has not yet been called, or the remove method has already been called after the last call to the next method

```
// Java program to demonstrate Iterator
```

```

import java.util.ArrayList;
import java.util.Iterator;

public class Test
{
    public static void main(String[] args)
    {
        ArrayList al = new ArrayList();

        for (int i = 0; i < 10; i++)
            al.add(i);

        System.out.println(al);

        // at beginning itr(cursor) will point to
        // index just before the first element in al
        Iterator itr = al.iterator();

        // checking the next element availability
        while (itr.hasNext())
        {
            // moving cursor to next element
            int i = (Integer)itr.next();

            // getting even elements one by one
            System.out.print(i + " ");

            // Removing odd elements
            if (i % 2 != 0)
                itr.remove();
        }
        System.out.println();
        System.out.println(al);
    }
}

```

Output:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
0 1 2 3 4 5 6 7 8 9
```

```
[0, 2, 4, 6, 8]
```

### Limitations of Iterator :

- Only forward direction iterating is possible.
- Replacement and addition of new element is not supported by Iterator.

### 3. ListIterator:

It is only applicable for List collection implemented classes like arraylist, linkedlist etc. It provides bi-directional iteration.

ListIterator must be used when we want to enumerate elements of List. This cursor has more functionality(methods) than iterator.

ListIterator object can be created by calling *listIterator()* method present in List interface.

```
// Here "l" is any List object, ltr is of type
```

```
// ListIterator interface and refers to "l"
```

```
ListIterator ltr = l.listIterator();
```

ListIterator interface extends Iterator interface. So all three methods of Iterator interface are available for ListIterator. In addition there are **six** more methods.

```
// Forward direction
```

```
// Returns true if the iteration has more elements
```

```
public boolean hasNext();
```

```
// same as next() method of Iterator
```

```
public Object next();
```

```
// Returns the next element index
```

```
// or list size if the list iterator
```

```
// is at the end of the list
```

```
public int nextIndex();
```

```
// Backward direction
```

```
// Returns true if the iteration has more elements
```

```
// while traversing backward
```

```
public boolean hasPrevious();
```

```
// Returns the previous element in the iteration
```

```
// and can throws NoSuchElementException
```

```
// if no more element present
```

```
public Object previous();
```

```
// Returns the previous element index
```

```
// or -1 if the list iterator is at the
```

```
// beginning of the list
```

```
public int previousIndex();
```

```
// Other Methods
```

```
// same as remove() method of Iterator
```

```
public void remove();
```

```
// Replaces the last element returned by
```

```
// next() or previous() with the specified element
```

```
public void set(Object obj);
```

```
// Inserts the specified element into the list at
```

```
// position before the element that would be returned
```

```
// by next(),
```

```
public void add(Object obj);
```

```
// Java program to demonstrate ListIterator
```

```
import java.util.ArrayList;
```

```
import java.util.ListIterator;
```

```
public class Test
```

```

{
    public static void main(String[] args)
    {
        ArrayList al = new ArrayList();
        for (int i = 0; i < 10; i++)
            al.add(i);

        System.out.println(al);

        // at beginning ltr(cursor) will point to
        // index just before the first element in al
        ListIterator ltr = al.listIterator();

        // checking the next element availability
        while (ltr.hasNext())
        {
            // moving cursor to next element
            int i = (Integer)ltr.next();

            // getting even elements one by one
            System.out.print(i + " ");

            // Changing even numbers to odd and
            // adding modified number again in
            // iterator
            if (i%2==0)
            {
                i++; // Change to odd
                ltr.set(i); // set method to change value
                ltr.add(i); // to add
            }
        }
        System.out.println();
        System.out.println(al);
    }
}

```

Output:

```

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
0 1 2 3 4 5 6 7 8 9
[1, 1, 1, 3, 3, 3, 5, 5, 5, 7, 7, 7, 9, 9, 9]

```

## Random access interface

### public interface **RandomAccess**

Marker interface used by `List` implementations to indicate that they support fast (generally constant time) random access. The primary purpose of this interface is to allow generic algorithms to alter their behavior to provide good performance when applied to either random or sequential access lists.

The best algorithms for manipulating random access lists (such as `ArrayList`) can produce quadratic behavior when applied to sequential access lists (such as `LinkedList`). Generic list algorithms are encouraged to check whether the given list is an instance of this interface before applying an algorithm that would provide poor performance if it were applied to a sequential access list, and to alter their behavior if necessary to guarantee acceptable performance.

It is recognized that the distinction between random and sequential access is often fuzzy. For example, some `List` implementations provide asymptotically linear access times if they get huge, but constant access times in practice. Such a `List` implementation should generally implement this interface. As a rule of thumb, a `List` implementation should implement this interface if, for typical instances of the class, this loop:

```
for (int i=0, n=list.size(); i < n; i++)
    list.get(i);
```

runs faster than this loop:

```
for (Iterator i=list.iterator(); i.hasNext(); )
    i.next();
```

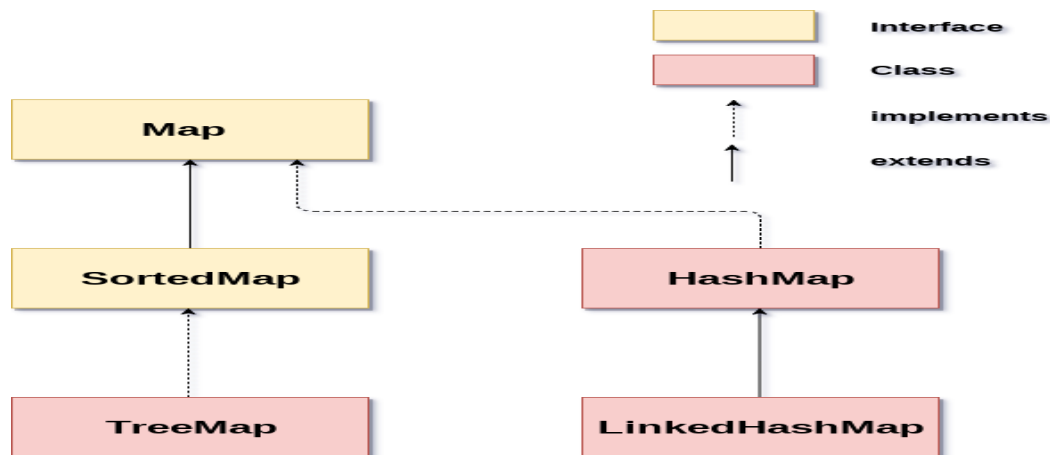
## Java Map Interface

A map contains values on the basis of key, i.e. key and value pair. Each key and value pair is known as an entry. A Map contains unique keys.

A Map is useful if you have to search, update or delete elements on the basis of a key.

## Java Map Hierarchy

There are two interfaces for implementing Map in java: Map and SortedMap, and three classes: HashMap, LinkedHashMap, and TreeMap. The hierarchy of Java Map is given below:



A Map doesn't allow duplicate keys, but you can have duplicate values. HashMap and LinkedHashMap allow null keys and values, but TreeMap doesn't allow any null key or value.

A Map can't be traversed, so you need to convert it into Set using *keySet()* or *entrySet()* method.

| Class                | Description                                                                                          |
|----------------------|------------------------------------------------------------------------------------------------------|
| <u>HashMap</u>       | HashMap is the implementation of Map, but it doesn't maintain any order.                             |
| <u>LinkedHashMap</u> | LinkedHashMap is the implementation of Map. It inherits HashMap class. It maintains insertion order. |

### TreeMap

TreeMap is the implementation of Map and SortedMap. It maintains ascending order.

## Useful methods of Map interface

| Method                                   | Description                                                                                           |
|------------------------------------------|-------------------------------------------------------------------------------------------------------|
| V put(Object key, Object value)          | It is used to insert an entry in the map.                                                             |
| void putAll(Map map)                     | It is used to insert the specified map in the map.                                                    |
| V putIfAbsent(K key, V value)            | It inserts the specified value with the specified key in the map only if it is not already specified. |
| V remove(Object key)                     | It is used to delete an entry for the specified key.                                                  |
| boolean remove(Object key, Object value) | It removes the specified values with the associated specified keys from the map.                      |
| Set keySet()                             | It returns the Set view containing all the keys.                                                      |
| Set<Map.Entry<K,V>> entrySet()           | It returns the Set view containing all the keys and values.                                           |
| void clear()                             | It is used to reset the map.                                                                          |

## Map.Entry Interface

Entry is the subinterface of Map. So we will be accessed it by Map.Entry name. It returns a collection-view of the map, whose elements are of this class. It provides methods to get key and value.

## Methods of Map.Entry interface

| Method     | Description                 |
|------------|-----------------------------|
| K getKey() | It is used to obtain a key. |

|                     |                                                                                       |
|---------------------|---------------------------------------------------------------------------------------|
| V getValue()        | It is used to obtain value.                                                           |
| int hashCode()      | It is used to obtain hashCode.                                                        |
| V setValue(V value) | It is used to replace the value corresponding to this entry with the specified value. |

## Java Map Example: Generic

```
import java.util.*;
class MapExample2{
public static void main(String args[]){
    Map<Integer,String> map=new HashMap<Integer,String>();
    map.put(100,"Amit");
    map.put(101,"Vijay");
    map.put(102,"Rahul");
    //Elements can traverse in any order
    for(Map.Entry m:map.entrySet()){
        System.out.println(m.getKey()+" "+m.getValue());
    }
}
}
```

Output:

```
102 Rahul
100 Amit
101 Vijay
```

## Comparator Interface in Java with Examples

Comparator interface is used to order the objects of user-defined classes. A comparator object is capable of comparing two objects of two different classes. Following function compare obj1 with obj2

**Syntax:**

```
public int compare(Object obj1, Object obj2):
```

**Method 1:** One obvious approach is to write our own sort() function using one of the standard algorithms. This solution requires rewriting the whole sorting code for different criterion like Roll No. and Name.

**Method 2:** Using comparator interface- Comparator interface is used to order the objects of user-defined class. This interface is present in java.util package and contains 2 methods compare(Object obj1, Object obj2) and equals(Object element).

Using comparator, we can sort the elements based on data members. For instance it may be on rollno, name, age or anything else.

Method of Collections class for sorting List elements is used to sort the elements of List by the given comparator.

```
// To sort a given list. ComparatorClass must implement  
// Comparator interface.  
public void sort(List list, ComparatorClass c)
```

## Legacy Classes

Early version of java did not include the Collections framework. It only defined several classes and interfaces that provide methods for storing objects. When Collections framework were added in J2SE 1.2, the original classes were reengineered to support the collection interface. These classes are also known as Legacy classes. All legacy classes and interface were redesign by JDK 5 to support Generics. In general, the legacy classes are supported because there is still some code that uses them.

The following are the legacy classes defined by **java.util** package

1. Dictionary
2. HashTable
3. Properties
4. Stack
5. Vector

There is only one legacy interface called **Enumeration**

### ***Enumeration interface***

1. **Enumeration** interface defines method to enumerate(obtain one at a time) through collection of objects.
2. This interface is superseded(replaced) by **Iterator** interface.
3. However, some legacy classes such as **Vector** and **Properties** defines several method in which **Enumeration** interface is used.
4. It specifies the following two methods

### ***Vector class***

1. **Vector** is similar to **ArrayList** which represents a dynamic array.

2. There are two differences between **Vector** and **ArrayList**. First, Vector is synchronized while ArrayList is not, and Second, it contains many legacy methods that are not part of the Collections Framework.
3. With the release of JDK 5, Vector also implements Iterable. This means that Vector is fully compatible with collections, and a Vector can have its contents iterated by the for-each loop.
4. Vector class has following four constructor

### ***Example of Vector***

```
import java.util.*;
public class Test
{
    public static void main(String[] args)
    {
        Vector ve = new Vector();
        ve.add(10);
        ve.add(20);
        ve.add(30);
        ve.add(40);
        ve.add(50);
        ve.add(60);

        Enumeration en = ve.elements();

        while(en.hasMoreElements())
        {
            System.out.println(en.nextElement());
        }
    }
}
```

### ***Hashtable class***

1. Like HashMap, Hashtable also stores key/value pair. However neither **keys** nor **values** can be **null**.
2. There is one more difference between **HashMap** and **Hashtable** that is Hashtable is synchronized while HashMap is not.

3. Hashtable has following four constructor

### ***Example of Hashtable***

```
import java.util.*;
class HashtableDemo
{
    public static void main(String args[])
    {
        Hashtable< String,Integer> ht = new Hashtable< String,Integer>();
        ht.put("a",new Integer(100));
        ht.put("b",new Integer(200));
        ht.put("c",new Integer(300));
        ht.put("d",new Integer(400));

        Set st = ht.entrySet();
        Iterator itr=st.iterator();
        while(itr.hasNext())
        {
            Map.Entry m=(Map.Entry)itr.next();
            System.out.println(itr.getKey()+" "+itr.getValue());
        }
    }
}
```

### ***Properties class***

1. **Properties** class extends **Hashtable** class.
2. It is used to maintain list of value in which both key and value are **String**
3. **Properties** class define two constructor
4. One advantage of **Properties** over **Hashtable** is that we can specify a default property that will be useful when no value is associated with a certain key.  
**Note:** In both cases, the property list is empty
5. In Properties class, you can specify a default property that will be returned if no value is associated with a certain key.

### ***Stack class***

1. Stack class extends Vector.

2. It follows last-in, first-out principle for the stack elements.
3. It defines only one default constructor

```
Stack() //This creates an empty stack
```

4. If you want to put an object on the top of the stack, call push() method. If you want to remove and return the top element, call pop() method. An EmptyStackException is thrown if you call pop() method when the invoking stack is empty.

### **Dictionary class**

1. Dictionary is an abstract class.
2. It represents a key/value pair and operates much like Map.
3. Although it is not currently deprecated, Dictionary is classified as obsolete, because it is fully superseded by Map class.

## StringTokenizer

The **java.util.StringTokenizer** class allows an application to break a string into tokens.

- This class is a legacy class that is retained for compatibility reasons although its use is discouraged in new code.
- Its methods do not distinguish among identifiers, numbers, and quoted strings.
- This class methods do not even recognize and skip comments.

## Class declaration

Following is the declaration for **java.util.StringTokenizer** class –

```
public class StringTokenizer  
    extends Object  
    implements Enumeration<Object>
```

## Class constructors

| Sr.No. | Constructor & Description                                                                               |
|--------|---------------------------------------------------------------------------------------------------------|
| 1      | <b>StringTokenizer(String str)</b><br><br>This constructor a string tokenizer for the specified string. |

|   |                                                                                                                                                        |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | <b>StringTokenizer(String str, String delim)</b><br><br>This constructor constructs string tokenizer for the specified string.                         |
| 3 | <b>StringTokenizer(String str, String delim, boolean returnDelims)</b><br><br>This constructor constructs a string tokenizer for the specified string. |

## Class methods

| Sr.No. | Method & Description                                                                                                                                                      |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1      | <b><u>int countTokens()</u></b><br><br>This method calculates the number of times that this tokenizer's nextToken method can be called before it generates an exception.  |
| 2      | <b><u>boolean hasMoreElements()</u></b><br><br>This method returns the same value as the hasMoreTokens method.                                                            |
| 3      | <b><u>boolean hasMoreTokens()</u></b><br><br>This method tests if there are more tokens available from this tokenizer's string.                                           |
| 4      | <b><u>Object nextElement()</u></b><br><br>This method returns the same value as the nextToken method, except that its declared return value is Object rather than String. |
| 5      | <b><u>String nextToken()</u></b><br><br>This method returns the next token from this string tokenizer.                                                                    |

6

**String nextToken(String delim)**

This method returns the next token in this string tokenizer's string.

```

1. import java.util.StringTokenizer;
2. public class Simple{
3.     public static void main(String args[]){
4.         StringTokenizer st = new StringTokenizer("my name is khan"," ");
5.         while (st.hasMoreTokens()) {
6.             System.out.println(st.nextToken());
7.         }
8.     }
9. }

```

```

Output:my
       name
       is
       khan

```

**BitSet**

**BitSet** is a class defined in the java.util package. It creates an array of bits represented by boolean values.

**Constructors:****BitSet class Constructors**

```

/          \
BitSet()    BitSet(int no_Of_Bits)

```

- **BitSet()** : A no-argument constructor to create an empty BitSet object.
- **BitSet(int no\_Of\_Bits)** : A one-constructor with an integer argument to create an instance of the BitSet class with an initial size of the integer argument representing the number of bits.

**Important Points :**

- The size of the array is flexible and can grow to accommodate additional bit as needed.
- As it is an array, the index is zero-based and the bit values can be accessed only by non-negative integers as an index .
- The default value of the BitSet is boolean false with a representation as 0 (off).
- Calling the clear method makes the bit values set to false.
- BitSet uses about 1 bit per boolean value.
- To access a specific value in the BitSet, the get method is used with an integer argument as an index.

```

// Java program illustrating Bitset Class constructors.
import java.util.*;
public class GFG
{
    public static void main(String[] args)
    {
        // Constructors of BitSet class
        BitSet bs1 = new BitSet();
        BitSet bs2 = new BitSet(6);
    }
}

```

```

        /* set is BitSet class method
           expalined in next articles */
        bs1.set(0);
        bs1.set(1);
        bs1.set(2);
        bs1.set(4);

        // assign values to bs2
        bs2.set(4);
        bs2.set(6);
        bs2.set(5);
        bs2.set(1);
        bs2.set(2);
        bs2.set(3);

        // Printing the 2 Bitsets
        System.out.println("bs1   : " + bs1);
        System.out.println("bs2   : " + bs2);
    }
}

```

## Date class in Java

The class Date represents a specific instant in time, with millisecond precision. The Date class of java.util package implements Serializable, Cloneable and Comparable interface. It provides constructors and methods to deal with date and time with java.

### Constructors

- **Date()** : Creates date object representing current date and time.
- **Date(long milliseconds)** : Creates a date object for the given milliseconds since January 1, 1970, 00:00:00 GMT.
- **Date(int year, int month, int date)**
- **Date(int year, int month, int date, int hrs, int min)**
- **Date(int year, int month, int date, int hrs, int min, int sec)**
- **Date(String s)**

```

// Java program to demonstrate constructors of Date
import java.util.*;

public class Main
{
    public static void main(String[] args)
    {
        Date d1 = new Date();
        System.out.println("Current date is " + d1);
        Date d2 = new Date(2323223232L);
        System.out.println("Date represented is "+ d2 );
    }
}

```

Output:

```
Current date is Tue Jul 12 18:35:37 IST 2016
```

## Calendar class in Java

Calendar class in Java is an abstract class that provides methods for converting date between a specific instant in time and a set of calendar fields such as MONTH, YEAR, HOUR, etc. It inherits Object class and implements the Comparable, Serializable, Cloneable interfaces.

As it is an Abstract class, so we cannot use a constructor to create an instance. Instead, we will have to use the static method `Calendar.getInstance()` to instantiate and implement a sub-class.

- `Calendar.getInstance()`: return a Calendar instance based on the current time in the default time zone with the default locale.
- `Calendar.getInstance(TimeZone zone)`
- `Calendar.getInstance(Locale aLocale)`
- `Calendar.getInstance(TimeZone zone, Locale aLocale)`

```
// Date getTime(): It is used to return a
```

```
// Date object representing this
```

```
// Calendar's time value.
```

```
import java.util.*;
```

```
public class Calendar1 {
```

```
    public static void main(String args[])
```

```
    {
```

```
        Calendar c = Calendar.getInstance();
```

```
        System.out.println("The Current Date is:" + c.getTime());
```

```
    }
```

```
}
```

### Output:

```
The Current Date is:Tue Aug 28 11:10:40 UTC 2018
```

## Java Timer class

The Java Timer class provides a facility for the thread to schedule tasks for future execution in a background thread. This class is thread-safe that means multiple class can share a single Timer object without using the external synchronization.

## Java Timer Methods

| SN | Method                                                                         | Description                                                                                                        |
|----|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1  | <u><a>cancel()</a></u>                                                         | It is used to stop the timer and remove the currently scheduled task.                                              |
| 2  | <u><a>Purge()</a></u>                                                          | It is used to cancel the tasks present in TimerTask Chain.                                                         |
| 3  | <u><a>scheduleAtFixedRate(TimerTask task, Date firstTime, long period)</a></u> | It is used to schedules the specified task for repeated fixed-rate execution, beginning at the specified time.     |
| 4  | <u><a>scheduleAtFixedRate(TimerTask task, long delay, long period)</a></u>     | It is used to schedules the specified task for repeated fixed-rate execution, beginning after the specified delay. |
| 5  | <u><a>schedule(TimerTask task, Date firstTime, long period)</a></u>            | It is used to schedules the specified task for repeated fixed-delay execution, beginning at the specified time.    |
| 6  | <u><a>schedule(TimerTask task, Date time)</a></u>                              | It is used to schedules the specified task for execution at the specified time.                                    |

```
import java.util.TimerTask;

public class RunMeTask extends TimerTask
{
    @Override
    public void run() {
        System.out.println("Run Me ~");
    }
}
```